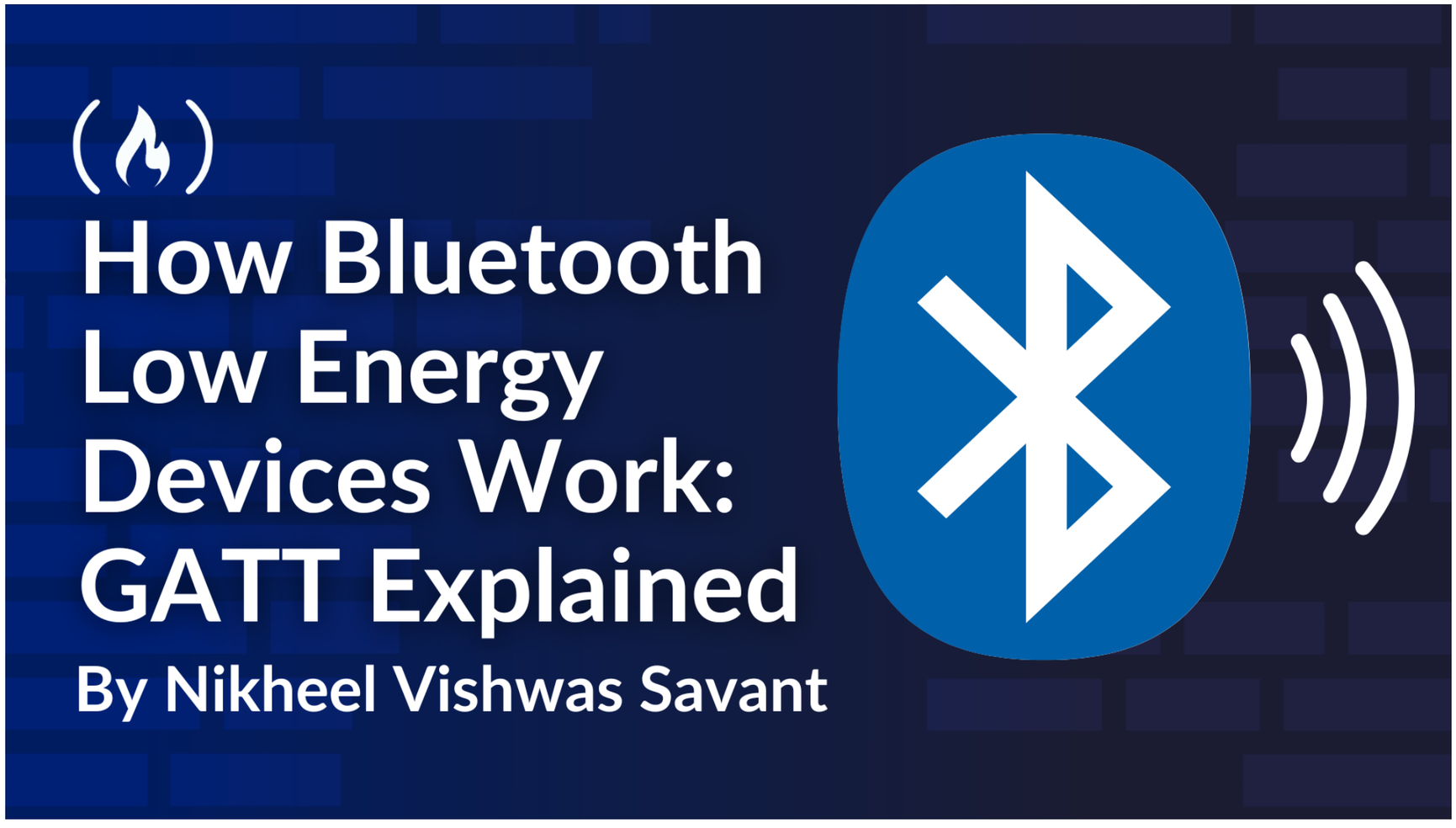
DECEMBER 3, 2025 / #BLUETOOTH GATT

How Bluetooth Low Energy Devices Work: GATT Services and Characteristics Explained

N Nikheel Vishwas Savant



Every time you check your smartwatch for heart rate, read the battery level of wireless earbuds, unlock a Bluetooth smart lock, or watch sensor data stream into an app, you are experiencing the result of GATT working quietly in the background.

GATT is the Generic Attribute Profile, and it provides the structure that makes Bluetooth Low Energy (BLE) devices exchange meaningful information. Without GATT, Bluetooth radios would simply move bits back and forth with no agreed format or interpretation. With GATT, devices can communicate in a predictable and understandable language.

Think of Bluetooth radios as two people speaking to each other in a room. The radio waves allow them to talk, but without a common language, the exchange is useless. GATT provides that common language. It defines the vocabulary, grammar, and sentence structure. Instead of random binary, we get clear messages like Heart Rate equals 78 bpm, Battery equals 92 percent, or Light Switch equals ON.

Because of GATT, devices from different manufacturers are able to interoperate. A Polar heart rate strap can connect to a Peloton bike. A Samsung phone can read temperature from a medical sensor. An Apple Watch can control Philips Hue smart lights. These devices do not share hardware, companies, or operating systems, yet they can cooperate because GATT defines a universal structure for exposing and accessing data.

Once you understand GATT, Bluetooth becomes far less mysterious. Communication becomes a matter of reading or writing values in a small

achievable, even for beginners.

In this article, we'll walk through GATT in depth. You'll learn how devices organize data into services and characteristics, how phones discover and read values, how notifications deliver real time updates, and how embedded and Android code interact with GATT. By the end, you'll be able to design a GATT database, understand BLE logs, and confidently build BLE applications.

Prerequisites

Before you continue, you should have a basic understanding of:

- What Bluetooth is at a high level (no deep protocol knowledge needed)
- How mobile apps connect to external devices (Android, iOS, or embedded)
- Very basic programming concepts (variables, functions, objects)

You'll also need:

- A smartphone or laptop with Bluetooth Low Energy support
- A BLE-compatible development board or device (optional, but helpful if you want to try the code examples)
- A BLE debugging/scanning app such as nRF Connect, LightBlue, or BLE Scanner

If you're completely new to BLE, don't worry – this article walks through each concept step-by-step.

- What is GATT?
- What Are Services and Why Do They Matter?
- What are Characteristics and How Do They Work?
- How to Design a GATT Profile for a Smart Plant Monitor
- Conclusion

What is GATT?

GATT stands for Generic Attribute Profile. It is the structured communication model used by Bluetooth Low Energy devices to exchange data in a clear and organized format.

GATT defines how data is stored, formatted, accessed, updated, and transmitted across BLE connections. Without GATT, Bluetooth devices would only exchange unstructured binary information that has no consistent meaning. With GATT, devices can share values such as battery percentage, heart rate, temperature readings, and status commands in a well-defined way.

GATT Client and Server Roles

All communication in BLE occurs between two roles. The GATT Server owns and exposes the data. The GATT Client requests, reads, writes, or subscribes to that data. The Server holds a database of values that the Client interacts with. A smartwatch usually acts as a GATT Server because it holds sensor values. A smartphone usually acts as a GATT Client because it retrieves that information.

These roles can switch depending on the task. For example, during a firmware update, the phone acts as the GATT Server providing firmware

Services, Characteristics, and UUIDs

The GATT Server stores its data in a structured database made up of Services and Characteristics. A Service is a container that groups related information. A Characteristic is a single data value inside a service.

For example, the Battery Service contains the Battery Level characteristic. The Heart Rate Service contains the Heart Rate Measurement characteristic.

All services and characteristics are identified using UUID values so that every device knows how to locate them. Standard Bluetooth SIG defined services such as Heart Rate and Battery use 16 bit UUIDs. Custom proprietary features use 128 bit UUIDs.

Example GATT Database Layout

Here is a conceptual breakdown of a simple GATT database layout:

```
Service: Battery Service (UUID 0x180F)
    Characteristic: Battery Level (UUID 0x2A19)
    Example value: 92 percent
```

Example: Reading a GATT Characteristic from Android

When a phone connects to a BLE device and acts as the client, it performs a sequence of steps. It connects to the device, discovers services, finds the characteristic of interest, and reads or subscribes to its value.

The following complete Java example shows an Android app acting as a **GATT Client**, discovering services and reading the battery level.

```
public class BleGattClientManager {

    private BluetoothGatt bluetoothGatt;

    private final BluetoothGattCallback gattCallback = new BluetoothGattCallback() {

        @Override
        public void onConnectionStateChange(BluetoothGatt gatt, int status, int newS
            if (newState == BluetoothProfile.STATE_CONNECTED) {
                Log.d(TAG, "Connected to device. Discovering services.");
                gatt.discoverServices();
            }
        }

        @Override
        public void onServicesDiscovered(BluetoothGatt gatt, int status) {
            UUID BATTERY_SERVICE_UUID =
                    UUID.fromString("0000180F-0000-1000-8000-00805F9B34FB");
            UUID BATTERY_LEVEL_UUID =
                    UUID.fromString("00002A19-0000-1000-8000-00805F9B34FB");

            BluetoothGattService service = gatt.getService(BATTERY_SERVICE_UUID);
            if (service != null) {
                BluetoothGattCharacteristic characteristic =
                        service.getCharacteristic(BATTERY_LEVEL_UUID);
                if (characteristic != null) {
                    gatt.readCharacteristic(characteristic);
                }
            }
        }

        @Override
        public void onCharacteristicRead(BluetoothGatt gatt,
                                         BluetoothGattCharacteristic characteristic
                                         int status) {
            if (status == BluetoothGatt.GATT_SUCCESS) {
                int batteryValue = characteristic.getIntValue(
                        BluetoothGattCharacteristic.FORMAT_UINT8, 0);
                Log.d(TAG, "Battery Level: " + batteryValue + " percent");
            }
        }
    };

    public void connect(Context context, BluetoothDevice device) {
        bluetoothGatt = device.connectGatt(context, false, gattCallback);
    }
}
```

class holds a `BluetoothGatt` object that represents the active BLE connection. The `BluetoothGattCallback` handles events during the connection lifecycle.

When the device connection state changes and the new state indicates that the device is connected, the callback triggers service discovery by calling `gatt.discoverServices()`.

After the services are discovered, the callback receives `onServicesDiscovered`, where two standard UUIDs are defined: the Battery Service with UUID `0000180F-0000-1000-8000-00805F9B34FB` and the Battery Level characteristic with UUID `00002A19-0000-1000-8000-00805F9B34FB`. The client retrieves the Battery Service using `gatt.getService`, then retrieves the Battery Level characteristic using `getCharacteristic`.

If both objects are found, the client calls `gatt.readCharacteristic`, which sends a read request to the server. When the server responds, `onCharacteristicRead` is invoked. If the response is successful, the characteristic value is extracted using `getIntValue` as an unsigned 8 bit integer at offset zero, producing a percentage from zero to one hundred. This value is printed to the log.

The `connect` method initiates the connection by calling `device.connectGatt`, which begins the communication and links all callbacks.

In summary, the flow is simple: connect to the device, discover services, locate the Battery Service, read the Battery Level characteristic, and print the result. This code shows the core pattern of how a BLE client interacts

Android as a GATT Server

The Android device can also act as a **GATT Server**. This is useful when the phone needs to expose its own characteristics that other BLE devices read or write, such as setup information, commands, or configuration data.

Below is a complete example of a custom GATT Server written in Java. It exposes a custom service and a custom characteristic that allows a BLE client to read and write values.

```
public class BleGattServerManager {

    private BluetoothGattServer gattServer;

    private final UUID SERVICE_UUID =
            UUID.fromString("12345678-1234-5678-1234-56789abcdef0");

    private final UUID CHARACTERISTIC_UUID =
            UUID.fromString("abcdef01-1234-5678-1234-56789abcdef0");

    private final BluetoothGattServerCallback serverCallback =
            new BluetoothGattServerCallback() {

        @Override
        public void onConnectionStateChange(BluetoothDevice device,
                                            int status,
                                            int newState) {
            Log.d(TAG, "Device connected: " + device.getAddress());
        }

        @Override
        public void onCharacteristicReadRequest(BluetoothDevice device,
                                                int requestId,
                                                int offset,
                                                BluetoothGattCharacteristic charact

            byte[] value = "HELLO_ANDROID_SERVER".getBytes(StandardCharsets.UTF_8);
            gattServer.sendResponse(device, requestId,
                    BluetoothGatt.GATT_SUCCESS, offset, value);
        }

        @Override
        public void onCharacteristicWriteRequest(BluetoothDevice device,
```

```
                                        int offset,
                                        byte[] value) {

            String received = new String(value, StandardCharsets.UTF_8);
            Log.d(TAG, "Client wrote value: " + received);

            gattServer.sendResponse(device, requestId,
                    BluetoothGatt.GATT_SUCCESS, offset, value);
        }
    };

    public void startServer(Context context) {
        BluetoothManager bluetoothManager =
                (BluetoothManager) context.getSystemService(Context.BLUETOOTH_SERVI(

        gattServer = bluetoothManager.openGattServer(context, serverCallback);

        BluetoothGattService customService =
                new BluetoothGattService(SERVICE_UUID,
                        BluetoothGattService.SERVICE_TYPE_PRIMARY);

        BluetoothGattCharacteristic customCharacteristic =
                new BluetoothGattCharacteristic(
                        CHARACTERISTIC_UUID,
                        BluetoothGattCharacteristic.PROPERTY_READ |
                        BluetoothGattCharacteristic.PROPERTY_WRITE,
                        BluetoothGattCharacteristic.PERMISSION_READ |
                        BluetoothGattCharacteristic.PERMISSION_WRITE
                );

            customService.addCharacteristic(customCharacteristic);
            gattServer.addService(customService);
        }
    }
```

This Java class implements a Bluetooth Low Energy GATT Server on Android, meaning it exposes a service and characteristic that another BLE device can read or write.

The class holds a `BluetoothGattServer` instance which is created when the server starts. Two UUIDs are defined, one for the custom service and one for the custom characteristic. The `BluetoothGattServerCallback` handles

When a client sends a read request on the characteristic, `onCharacteristicReadRequest` responds by sending back a static string value, in this case the bytes of the text `HELLO_ANDROID_SERVER`, using `sendResponse` with a success status.

When the client writes data to the characteristic, `onCharacteristicWriteRequest` converts the incoming byte array to a string, logs what was written, and returns a success response to acknowledge that the server accepted the new value.

The `startServer` method initializes the GATT Server by requesting the `BluetoothManager`, opening the server, creating a custom primary service, and adding a characteristic to that service with both read and write properties and permissions. The service is then registered on the server through `addService`, which makes it available to any BLE client that connects.

In summary, this code demonstrates how an Android device can behave like a BLE peripheral and expose a custom readable and writable characteristic that other devices can interact with. This forms the foundation for features like configuration setup, provisioning, remote control commands, or device to device communication.

This example shows that GATT is simply a structured database of readable and writable values that represent meaningful application behavior. Whether the task is battery level reporting, real time health monitoring, remote control of smart devices, or secure provisioning, the exchange always follows this same pattern.

Understanding GATT at this level is the foundation for all Bluetooth Low

What Are Services and Why Do They Matter?

Services as Capability Containers

A Service in GATT is a logical container that groups related data. A single Bluetooth device can expose many services, and each service focuses on one capability or functional category.

For example, a smartwatch may expose a Heart Rate Service, a Battery Service, a Current Time Service, and a Device Information Service. A smart bulb may expose a Lighting Control Service with characteristics to change brightness and color. A medical thermometer may expose a Health Thermometer Service that continuously streams temperature values.

Services exist to separate different categories of information so that any client device can immediately understand what functions are available and how to interact with them.

A service does not hold raw values itself. Instead, it organizes characteristics, which contain the actual data elements. The service only defines the grouping and the type of behavior associated with that group. This design makes BLE communication extremely scalable. Applications only need to know which service to target, then they can discover and manipulate the characteristics inside it.

Standard vs Custom Services

Bluetooth SIG defines many standard services for interoperability. For example, the Battery Service exposes battery level in a characteristic called Battery Level. The Heart Rate Service exposes heart rate values so that any fitness application can subscribe to it. These standard services allow

Anyone building a custom device can also define their own original service using a 128 bit UUID. The structure is the same whether it is standard or custom.

Example: A Device with Multiple Services

Below is an example representation of a device that exposes two services at the same time:

```
Service: Battery Service (UUID 0x180F)
    Characteristic: Battery Level (UUID 0x2A19)
    Current value: 92 percent

Service: Heart Rate Service (UUID 0x180D)
    Characteristic: Heart Rate Measurement (UUID 0x2A37)
    Current value: 78 bpm
```

An Android phone acting as a client can discover these services and then interact with characteristics inside them. Discovery is always the first step after establishing a BLE connection.

Discovering Services in Android

The following example shows how to list all available services and log them in Java.

```
@Override
public void onServicesDiscovered(BluetoothGatt gatt, int status) {
    for (BluetoothGattService service : gatt.getServices()) {
        Log.d(TAG, "Service discovered: " + service.getUuid().toString());
    }
}
```

discovering all services exposed by the remote GATT server. When a connection is established, the client initiates a service discovery procedure, and once the server responds with the complete list of available services, the Android stack triggers `onServicesDiscovered`.

Inside this callback, the code iterates through every `BluetoothGattService` returned by `gatt.getServices()`, which represents all services implemented by the connected device. For each service in that list, the code prints its UUID to the log. This output helps developers inspect what services exist on the device, confirm that expected services such as Heart Rate, Battery, or a custom service are present, and identify the correct UUIDs needed for reading or writing characteristics later.

This method is especially useful during development or debugging, because it allows you to verify that a device correctly exposes its GATT database and that the client can access the list of services before attempting to interact with any characteristics.

Notifications for Continuously Changing Data

Once the service is found, the next step is to read or subscribe to its characteristics. Some characteristics contain static or rarely changing values, which makes direct reads appropriate. Others, such as heart rate or temperature, change continuously, and should use notifications.

Why Notifications Matter in Services

Notifications allow a device to receive updates automatically whenever the value changes instead of repeatedly reading the characteristic. This reduces energy usage and latency, which is essential for wearables and sensors.

Rate Measurement characteristic.

```
private void enableHeartRateNotifications(BluetoothGatt gatt) {

    UUID HEART_RATE_SERVICE_UUID =
            UUID.fromString("0000180D-0000-1000-8000-00805F9B34FB");
    UUID HEART_RATE_MEASUREMENT_UUID =
            UUID.fromString("00002A37-0000-1000-8000-00805F9B34FB");

    BluetoothGattService service = gatt.getService(HEART_RATE_SERVICE_UUID);
    if (service != null) {
        BluetoothGattCharacteristic characteristic =
                service.getCharacteristic(HEART_RATE_MEASUREMENT_UUID);

        if (characteristic != null) {
            gatt.setCharacteristicNotification(characteristic, true);

            BluetoothGattDescriptor descriptor =
                    characteristic.getDescriptor(
                            UUID.fromString("00002902-0000-1000-8000-00805F9B34FB")
                    );

            if (descriptor != null) {
                descriptor.setValue(BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALU
                gatt.writeDescriptor(descriptor);
            }
        }
    }
}
```

Enabling Notifications in Android

This method enables notifications for the Heart Rate Measurement characteristic so that the device can push new values automatically whenever the heart rate changes.

It begins by defining the UUIDs for the Heart Rate Service and the Heart Rate Measurement characteristic. Using `gatt.getService`, it retrieves the Heart Rate Service from the connected device. If that service exists, it locates the Heart Rate Measurement characteristic within it. Once the

the client to receive asynchronous updates.

But enabling local notifications is not enough. The BLE specification requires writing to a special descriptor called the Client Characteristic Configuration Descriptor, identified by UUID `00002902-0000-1000-8000-00805F9B34FB`, so that the remote device also knows the client wants updates. The method retrieves this descriptor, sets its value to `ENABLE_NOTIFICATION_VALUE`, and writes it using `writeDescriptor`, which sends a request over the air to the server telling it to start sending notifications.

After this sequence completes, updates begin arriving automatically in the `onCharacteristicChanged` callback whenever the heart rate changes, without needing repeated read requests.

This is the preferred BLE pattern for continuous sensor data such as heart rate, temperature, step count, or motion values because it saves power and provides real time responsiveness.

When the device begins sending notifications, updates are received in the callback below. The values arrive whenever they change, making this very efficient for streaming.

```
@Override
public void onCharacteristicChanged(BluetoothGatt gatt,
                                    BluetoothGattCharacteristic characteristic) {

    UUID HEART_RATE_MEASUREMENT_UUID =
            UUID.fromString("00002A37-0000-1000-8000-00805F9B34FB");

    if (characteristic.getUuid().equals(HEART_RATE_MEASUREMENT_UUID)) {
        int flag = characteristic.getProperties();
        int format;
```

```
            }
            int heartRate = characteristic.getIntValue(format, 1);
            Log.d(TAG, "Heart Rate: " + heartRate + " bpm");
        }
    }
```

This method is called automatically whenever the Bluetooth device sends a notification indicating that the value of a characteristic has changed.

In this example, the method handles updates to the Heart Rate Measurement characteristic. The code first checks whether the characteristic that triggered the callback matches the Heart Rate Measurement UUID `00002A37-0000-1000-8000-00805F9B34FB`, ensuring that only relevant updates are processed.

The heart rate data can be encoded in either one or two bytes, depending on the flags defined in the characteristic properties. The method reads these flags and determines the correct format to use when decoding the heart rate value. If the least significant bit is set, the heart rate uses a 16 bit format. Otherwise, it uses a single 8 bit format.

After selecting the appropriate format, the method extracts the heart rate value using `getIntValue`, beginning at offset 1 because byte 0 contains the flags.

Finally, the value is printed to the log as beats per minute. This method is typically called repeatedly, such as once per second, so the log receives live heart rate updates in real time.

This approach demonstrates how notifications deliver continuous sensor data without repeatedly polling the device, which reduces latency and

This example demonstrates how the client retrieves real time sensor data using subscription instead of polling. The same mechanism is used for air quality sensors, smart home lighting brightness notifications, industrial temperature monitors, and more.

To summarize this section, a Service is a structured container that organizes related data and is essential for exposing abilities and functionality over BLE. Understanding how to discover and interact with services is the first major step toward building or debugging real Bluetooth applications.

What Are Characteristics and How Do They Work?

The Role of a Characteristic

If a Service is a folder, then a Characteristic is a file inside that folder that actually holds the content. In GATT, the Characteristic is where the real data lives. Almost everything your application cares about will eventually be read from, written to, or subscribed to on a characteristic.

The Four Parts of a Characteristic

A characteristic is more than just a single number. It has four important parts. First, it has a UUID that identifies what it represents. It also has a value that stores the actual bytes. Then it has properties that describe what operations are allowed, such as read, write, or notify. And finally, it has permissions that control who can access it and under what security level. Understanding these pieces is the key to working confidently with BLE.

The UUID tells you what kind of data is inside the characteristic. For

one hundred. A Heart Rate Measurement characteristic uses UUID 0x2A37 and packs heart rate and flags into a structured format. Custom characteristics use 128 bit UUIDs that developers define themselves.

The value is simply a sequence of bytes. On the wire, Bluetooth does not know about integers, floats, or strings. It only sees bytes. On the Android side, the `BluetoothGattCharacteristic` class helps interpret those bytes as different types. It provides helper methods such as `getIntValue`, `getFloatValue`, and `getStringValue` so that you can decode the data more easily.

The properties of a characteristic describe what kind of operations the client can perform. The most common properties are Read, Write, Notify, and Indicate.

Read means a client can ask the server to return the current value. Write means a client can send a new value to the server. Notify means the server can send updates to the client whenever the value changes. Indicate is similar to Notify, but with an extra confirmation. A characteristic may have one or many properties combined.

Permissions are related but slightly different. They focus on access control and security. For example, a characteristic may require encryption or authenticated pairing before it can be read or written. The Android `BluetoothGattCharacteristic` object contains these permission flags so that the stack enforces them correctly.

Example: Defining a Custom LED Characteristic

Let’s walk through a concrete example. Imagine a custom device that exposes a characteristic to control an LED state. The LED should be either

On the Android GATT Server side, you would define such a characteristic like this:

```
UUID SERVICE_UUID =
        UUID.fromString("12345678-1234-5678-1234-56789abcdef0");
UUID LED_CHAR_UUID =
        UUID.fromString("abcdef01-1234-5678-1234-56789abcdef0");

BluetoothGattService ledService =
        new BluetoothGattService(SERVICE_UUID,
                BluetoothGattService.SERVICE_TYPE_PRIMARY);

BluetoothGattCharacteristic ledCharacteristic =
        new BluetoothGattCharacteristic(
                LED_CHAR_UUID,
                BluetoothGattCharacteristic.PROPERTY_READ |
                BluetoothGattCharacteristic.PROPERTY_WRITE,
                BluetoothGattCharacteristic.PERMISSION_READ |
                BluetoothGattCharacteristic.PERMISSION_WRITE
        );

// Initial value
ledCharacteristic.setValue("OFF".getBytes(StandardCharsets.UTF_8));
ledService.addCharacteristic(ledCharacteristic);
gattServer.addService(ledService);
```

This code defines a custom GATT Service and a custom GATT Characteristic on an Android device acting as a Bluetooth Low Energy GATT Server.

Two UUIDs are created using `UUID.fromString`, one representing the custom service and the other representing the characteristic that belongs to that service. A new `BluetoothGattService` instance is then created, marked as a primary service to indicate that it is a main functional component rather than a secondary helper service.

remote BLE client. The property flags indicate that a client can request the current value and can also send updates, and the permission flags define that both operations are permitted.

The characteristic is given an initial value of the string `"OFF"` encoded as bytes, which might represent the current state of a remote controlled LED, device mode, or some other configuration setting.

The characteristic is then added to the service, and finally the fully defined service is added to the GATT server using `gattServer.addService`, making it visible to any BLE client that connects.

At this point, another device can read the value `"OFF"` or write a new value such as `"ON"`, which the server could then use to trigger real behavior, such as toggling actual hardware.

Handling Reads and Writes on the Server

Server-Side Handlers

On the GATT Server side, you must also respond to read and write requests. This happens inside `BluetoothGattServerCallback`.

```
private final BluetoothGattServerCallback serverCallback =
        new BluetoothGattServerCallback() {

    @Override
    public void onCharacteristicReadRequest(BluetoothDevice device,
                                            int requestId,
                                            int offset,
                                            BluetoothGattCharacteristic characteris

        byte[] currentValue = characteristic.getValue();
        gattServer.sendResponse(device,
                requestId,
```

```
@Override
public void onCharacteristicWriteRequest(BluetoothDevice device,
                                         int requestId,
                                         BluetoothGattCharacteristic characteri
                                         boolean preparedWrite,
                                         boolean responseNeeded,
                                         int offset,
                                         byte[] value) {

    String received = new String(value, StandardCharsets.UTF_8);
    Log.d(TAG, "LED characteristic write: " + received);

    characteristic.setValue(value);

    if ("ON".equalsIgnoreCase(received)) {
        turnLedOn();
    } else if ("OFF".equalsIgnoreCase(received)) {
        turnLedOff();
    } else {
        Log.e(TAG, "Unhandled case!");
        return;
    }

        if (responseNeeded) {
            gattServer.sendResponse(device,
                    requestId,
                    BluetoothGatt.GATT_SUCCESS,
                    offset,
                    value);
        }
    }
};
```

This callback handles read and write requests coming from a remote BLE client interacting with the custom LED characteristic on the GATT Server.

When a client performs a read operation, `onCharacteristicReadRequest` is triggered. The method retrieves the current stored value from the characteristic using `getValue()` and returns it to the client by calling `sendResponse` with a success status. This means whatever value was last set, such as `"ON"` or `"OFF"`, is sent back to the requesting device.

is called. The method converts the incoming byte array into a string so that the server can interpret the command. It logs the received text, sets the new value into the characteristic using `setValue`, and then checks whether the string equals `"ON"` or `"OFF"`. Depending on the value, it calls either `turnLedOn()` or `turnLedOff()`, which would typically control real hardware or trigger an action inside the application.

If the client requested a response, the server sends back a confirmation by calling `sendResponse` with `GATT_SUCCESS`, acknowledging that the write completed successfully. This callback demonstrates how interactive BLE control works: the server receives a command, updates internal state, performs a real action, and reports status back to the client.

Here, the server reads whatever value is stored in the characteristic upon a read request and sends it back to the client. When the client writes a new value, the server decodes the bytes as a string and updates internal state, including physical behavior like toggling the LED.

Reading and Writing from the Client

Client-Side Handlers

On the client side, a typical Android app needs to read and write to this same characteristic. The code for the client looks similar to what we saw in earlier sections, but now it uses the custom UUIDs.

Reading a Custom Characteristic (Client)

Reading the LED state from the client:

```
public void readLedState(BluetoothGatt gatt) {
    UUID SERVICE_UUID =
```

```
    BluetoothGattService service = gatt.getService(SERVICE_UUID);
    if (service != null) {
        BluetoothGattCharacteristic ledChar = service.getCharacteristic(LED_CHAR_UU
        if (ledChar != null) {
            gatt.readCharacteristic(ledChar);
        }
    }
}

@Override
public void onCharacteristicRead(BluetoothGatt gatt,
                                 BluetoothGattCharacteristic characteristic,
                                 int status) {
    if (status == BluetoothGatt.GATT_SUCCESS) {
        UUID LED_CHAR_UUID =
                UUID.fromString("abcdef01-1234-5678-1234-56789abcdef0");
        if (LED_CHAR_UUID.equals(characteristic.getUuid())) {
            String value = new String(characteristic.getValue(), StandardCharsets.U
            Log.d(TAG, "LED state is: " + value);
        }
    }
}
```

This code shows how a Bluetooth Low Energy client reads the current state of a custom LED characteristic from a GATT server. The `readLedState` method begins by defining the UUIDs for the custom service and the LED characteristic so that the client knows exactly where to look inside the server's GATT database.

It retrieves the service using `gatt.getService`, and if the service exists, it retrieves the LED characteristic using `getCharacteristic`. If that characteristic is found, the client calls `readCharacteristic`, which sends a read request to the remote device over BLE. Once the server responds, the callback method `onCharacteristicRead` is triggered.

This method first checks that the read was successful by confirming that the status equals `GATT_SUCCESS`. It then verifies that the characteristic

This flow demonstrates how a BLE client reads stored values from a peripheral device and responds when the server returns the data, forming the basis for real world interactions such as checking the status of a smart light, a switch, or any sensor value exposed through a custom characteristic.

Writing to a Custom Characteristic (Client)

Writing a new LED state from the client:

```
public void writeLedState(BluetoothGatt gatt, String newState) {
    UUID SERVICE_UUID =
            UUID.fromString("12345678-1234-5678-1234-56789abcdef0");
    UUID LED_CHAR_UUID =
            UUID.fromString("abcdef01-1234-5678-1234-56789abcdef0");
    BluetoothGattService service = gatt.getService(SERVICE_UUID);
    if (service != null) {
        BluetoothGattCharacteristic ledChar = service.getCharacteristic(LED_CHAR_UU
        if (ledChar != null) {
            ledChar.setValue(newState.getBytes(StandardCharsets.UTF_8));
            gatt.writeCharacteristic(ledChar);
        }
    }
}

@Override
public void onCharacteristicWrite(BluetoothGatt gatt,
                                  BluetoothGattCharacteristic characteristic,
                                  int status) {
    if (status == BluetoothGatt.GATT_SUCCESS) {
        Log.d(TAG, "LED state write completed");
    }
}
```

This code demonstrates how a Bluetooth Low Energy client sends a command to update the LED state on a GATT server device.

connected GATT server. If the service is found, it accesses the LED characteristic inside it.

Once the characteristic is obtained, the new LED state, which will typically be the string `"ON"` or `"OFF"`, is converted into a byte array and placed into the characteristic with `setValue`. The method then calls `writeCharacteristic`, which sends a write request to the remote device to update the stored value.

When the server processes the write and returns a response, the callback method `onCharacteristicWrite` executes. If the status indicates success, the code logs that the write completed. At this point, the server code on the other side can take action based on the new state, such as turning a real LED on or off.

This flow illustrates how clients modify values on a BLE peripheral and how acknowledgment is handled once the operation finishes, forming a typical example of device control over GATT.

This combination of server definitions and client interactions shows how characteristics are the real workhorses of GATT. Every meaningful piece of data flows through them. Reads, writes, and notifications all operate at the characteristic level.

Notifications and CCCD

Notifications are simply a special property on a characteristic. When enabled, the server can push new values to the client without the client asking every time. To support notifications, a characteristic needs the Notify property and usually a descriptor called the Client Characteristic Configuration Descriptor, often referred to as CCCD, with UUID 0x2902

`notifyCharacteristicChanged`. On the client side, you set characteristic notification to true and write the descriptor with `ENABLE_NOTIFICATION_VALUE`. The code pattern is almost identical regardless of the type of data, which makes it easy to reuse once you understand it.

By this point, you can see that a characteristic is not just a static field. It is a complete unit of behavior. It defines what data exists, how it is represented, who can access it, and how it updates. Once you're comfortable designing characteristics and manipulating them in Java, you're very close to mastering practical BLE development.

How to Design a GATT Profile for a Smart Plant Monitor

To make GATT feel real, let's design a complete profile for a simple but realistic device: a smart plant monitor.

Imagine a small BLE sensor that you stick into a flower pot. It measures soil moisture, reports its own battery level, and allows you to configure how often it sends updates. A phone app connects to it, reads the current moisture level, shows the battery percentage, and lets the user adjust the reporting interval.

We'll design both sides in terms of GATT. First, we'll decide which services and characteristics we need. Then, we'll see how an Android device can act as a client. For teaching purposes, we'll also show how Android could act as the server, although in a real product the plant monitor would normally be an embedded device.

1. Soil moisture percentage – this is dynamic sensor data.
2. Battery level – this is standard, so we can reuse the Battery Service.
3. Reporting interval configuration – this is a setting that the client writes and the device uses.

We can express this with one custom service plus the standard Battery Service.

Profile plan:

```
Custom Service: Plant Monitor Service
    UUID: 12345678-1234-5678-1234-56789abc0001

    Characteristic: Soil Moisture
        UUID: 12345678-1234-5678-1234-56789abc0002
        Properties: Read, Notify
        Permissions: Read
        Format: uint8 (0 to 100 percentage)

    Characteristic: Reporting Interval
        UUID: 12345678-1234-5678-1234-56789abc0003
        Properties: Read, Write
        Permissions: Read, Write
        Format: uint16 (seconds)

Standard Service: Battery Service
    UUID: 0000180F-0000-1000-8000-00805F9B34FB

    Characteristic: Battery Level
        UUID: 00002A19-0000-1000-8000-00805F9B34FB
        Properties: Read, Notify (optional)
        Permissions: Read
        Format: uint8 (0 to 100 percentage)
```

Now we know exactly what exists inside the device. A client can connect

Implementing the GATT Server

In a real hardware product, the plant monitor would be written in embedded C or C++. But for learning, we can simulate the server on Android itself. This'll help you understand how the server side works.

First, we'll create the services and characteristics.

```
public class PlantMonitorGattServer {

    private BluetoothGattServer gattServer;

    private final UUID PLANT_SERVICE_UUID =
            UUID.fromString("12345678-1234-5678-1234-56789abc0001");
    private final UUID MOISTURE_CHAR_UUID =
            UUID.fromString("12345678-1234-5678-1234-56789abc0002");
    private final UUID INTERVAL_CHAR_UUID =
            UUID.fromString("12345678-1234-5678-1234-56789abc0003");

    private final UUID BATTERY_SERVICE_UUID =
            UUID.fromString("0000180F-0000-1000-8000-00805F9B34FB");
    private final UUID BATTERY_LEVEL_UUID =
            UUID.fromString("00002A19-0000-1000-8000-00805F9B34FB");

    // Simulated state
    private int currentMoisture = 55;      // percentage
    private int reportingIntervalSec = 60; // seconds
    private int batteryLevel = 87;         // percentage

    private BluetoothGattCharacteristic moistureCharacteristic;
    private BluetoothGattCharacteristic intervalCharacteristic;
    private BluetoothGattCharacteristic batteryLevelCharacteristic;
```

This class represents a Bluetooth Low Energy GATT Server that pretends to be a smart plant monitor device. It holds a `BluetoothGattServer` instance that will expose services and characteristics to any BLE client that connects.

and its characteristics, as well as the standard Battery Service and Battery Level characteristic.

The custom Plant Monitor Service has two characteristics: one for soil moisture and one for the reporting interval. The Battery Service uses the standard UUIDs defined by the Bluetooth SIG so that any client can recognize and parse it.

The class also keeps some simulated internal state: `currentMoisture` starts at 55 percent, `reportingIntervalSec` is set to 60 seconds, and `batteryLevel` is set to 87 percent. These values act like sensor readings and configuration stored inside the device.

Finally, it declares three `BluetoothGattCharacteristic` fields that will later point to the actual moisture, interval, and battery level characteristics once they are created and added to their respective services.

These fields make it easy for the server to update values and send notifications later – for example, when moisture changes or when the battery level drops.

Next, we'll set up the server and define the services and characteristics.

```
public void startServer(Context context) {
    BluetoothManager bluetoothManager =
            (BluetoothManager) context.getSystemService(Context.BLUETOOTH_SERVI(

    gattServer = bluetoothManager.openGattServer(context, serverCallback);

    // Plant Monitor Service
    BluetoothGattService plantService =
            new BluetoothGattService(
                    PLANT_SERVICE_UUID,
```

```
        MOISTURE_CHAR_UUID,
        BluetoothGattCharacteristic.PROPERTY_READ |
        BluetoothGattCharacteristic.PROPERTY_NOTIFY,
        BluetoothGattCharacteristic.PERMISSION_READ
);

intervalCharacteristic = new BluetoothGattCharacteristic(
        INTERVAL_CHAR_UUID,
        BluetoothGattCharacteristic.PROPERTY_READ |
        BluetoothGattCharacteristic.PROPERTY_WRITE,
        BluetoothGattCharacteristic.PERMISSION_READ |
        BluetoothGattCharacteristic.PERMISSION_WRITE
);

// Set initial values
moistureCharacteristic.setValue(new byte[]{(byte) currentMoisture});
intervalCharacteristic.setValue(intToTwoBytes(reportingIntervalSec));

// For notifications, add CCCD descriptor
BluetoothGattDescriptor moistureCccd = new BluetoothGattDescriptor(
        UUID.fromString("00002902-0000-1000-8000-00805F9B34FB"),
        BluetoothGattDescriptor.PERMISSION_READ |
        BluetoothGattDescriptor.PERMISSION_WRITE
);
moistureCharacteristic.addDescriptor(moistureCccd);

plantService.addCharacteristic(moistureCharacteristic);
plantService.addCharacteristic(intervalCharacteristic);

// Battery Service
BluetoothGattService batteryService =
        new BluetoothGattService(
                BATTERY_SERVICE_UUID,
                BluetoothGattService.SERVICE_TYPE_PRIMARY
        );

batteryLevelCharacteristic = new BluetoothGattCharacteristic(
        BATTERY_LEVEL_UUID,
        BluetoothGattCharacteristic.PROPERTY_READ |
        BluetoothGattCharacteristic.PROPERTY_NOTIFY,
        BluetoothGattCharacteristic.PERMISSION_READ
);

batteryLevelCharacteristic.setValue(new byte[]{(byte) batteryLevel});

BluetoothGattDescriptor batteryCccd = new BluetoothGattDescriptor(
        UUID.fromString("00002902-0000-1000-8000-00805F9B34FB"),
        BluetoothGattDescriptor.PERMISSION_READ |
        BluetoothGattDescriptor.PERMISSION_WRITE
);
```

```
        gattServer.addService(plantService);
        gattServer.addService(batteryService);
    }

    private byte[] intToTwoBytes(int value) {
        byte[] data = new byte[2];
        data[0] = (byte) (value & 0xFF);
        data[1] = (byte) ((value >> 8) & 0xFF);
        return data;
    }
```

This method starts the Bluetooth Low Energy GATT server and builds the full GATT database for the smart plant monitor.

It first obtains the `BluetoothManager` from the Android system and uses it to open a `BluetoothGattServer`, passing in a callback that will handle read, write, and notification events from connected clients.

Then it creates the custom Plant Monitor Service using the `PLANT_SERVICE_UUID` and marks it as a primary service.

Inside this service it defines two characteristics. The moisture characteristic is created with the `MOISTURE_CHAR_UUID` and given the properties Read and Notify, meaning a client can read the current soil moisture and also subscribe to notifications when it changes. It is read only, so it uses a read permission. The reporting interval characteristic is created with the `INTERVAL_CHAR_UUID` and uses both Read and Write properties so that a client can check the current interval and update it. It uses both read and write permissions.

The code sets the initial values for these characteristics: the moisture characteristic gets the current moisture percentage stored as a single byte, and the interval characteristic gets a two byte representation of the

To allow notifications for moisture, it adds a Client Characteristic Configuration Descriptor (CCCD) with a standard UUID `0x2902` and read or write permissions, then attaches this descriptor to the moisture characteristic. Both characteristics are added to the plant service.

Next, the method creates the standard Battery Service as another primary service using the well-known battery UUID. It defines the Battery Level characteristic with read and notify properties and read permission.

The initial battery level is stored as a single byte. Just like with moisture, it adds a CCCD descriptor to support notifications and attaches it to the battery characteristic. The battery characteristic is then added to the battery service.

Finally, the method registers both the plant service and the battery service with the GATT server using `addService`, which makes them visible to any BLE client that connects. As a small utility, the `intToTwoBytes` method at the end converts a 16 bit integer into a two element byte array with the least significant byte first, which is a common way to encode integers in BLE characteristics.

Now we'll implement the callback to handle read, write, and notification logic.

```
    private final BluetoothGattServerCallback serverCallback =
            new BluetoothGattServerCallback() {

        @Override
        public void onConnectionStateChange(BluetoothDevice device,
                                            int status,
                                            int newState) {
```

```
@Override
public void onCharacteristicReadRequest(BluetoothDevice device,
                                        int requestId,
                                        int offset,
                                        BluetoothGattCharacteristic charact

    if (characteristic.getUuid().equals(MOISTURE_CHAR_UUID)) {
        moistureCharacteristic.setValue(new byte[]{(byte) currentMoisture})
        gattServer.sendResponse(device, requestId,
                BluetoothGatt.GATT_SUCCESS, offset,
                moistureCharacteristic.getValue());
    } else if (characteristic.getUuid().equals(INTERVAL_CHAR_UUID)) {
        intervalCharacteristic.setValue(intToTwoBytes(reportingIntervalSec)
        gattServer.sendResponse(device, requestId,
                BluetoothGatt.GATT_SUCCESS, offset,
                intervalCharacteristic.getValue());
    } else if (characteristic.getUuid().equals(BATTERY_LEVEL_UUID)) {
        batteryLevelCharacteristic.setValue(new byte[]{(byte) batteryLevel}
        gattServer.sendResponse(device, requestId,
                BluetoothGatt.GATT_SUCCESS, offset,
                batteryLevelCharacteristic.getValue());
    } else {
        gattServer.sendResponse(device, requestId,
                BluetoothGatt.GATT_FAILURE, offset, null);
    }
}

@Override
public void onCharacteristicWriteRequest(BluetoothDevice device,
                                         int requestId,
                                         BluetoothGattCharacteristic charac
                                         boolean preparedWrite,
                                         boolean responseNeeded,
                                         int offset,
                                         byte[] value) {

    if (characteristic.getUuid().equals(INTERVAL_CHAR_UUID)) {
        int newInterval = ((value[1] & 0xFF) << 8) | (value[0] & 0xFF);
        reportingIntervalSec = newInterval;
        Log.d(TAG, "New reporting interval: " + reportingIntervalSec + " se

        intervalCharacteristic.setValue(value);
    }

    if (responseNeeded) {
        gattServer.sendResponse(device, requestId,
                BluetoothGatt.GATT_SUCCESS, offset, value);
    }
}
```

```
                boolean preparedWrite,
                boolean responseNeeded,
                int offset,
                byte[] value) {

            if (descriptor.getCharacteristic().getUuid().equals(MOISTURE_CHAR_UUID)
                Log.d(TAG, "Moisture notifications enabled");
            } else if (descriptor.getCharacteristic().getUuid().equals(BATTERY_LEVEI
                Log.d(TAG, "Battery notifications enabled");
            }

            if (responseNeeded) {
                gattServer.sendResponse(device, requestId,
                        BluetoothGatt.GATT_SUCCESS, offset, value);
            }
        }
    };
}
```

This callback handles all the important server side events for the smart plant monitor GATT Server, including connection changes, characteristic reads and writes, and descriptor writes for notifications.

When a device connects or disconnects, `onConnectionStateChange` is called and simply logs the new connection state so you can see when a client appears or disappears. The core logic lives in `onCharacteristicReadRequest`, which is invoked whenever a BLE client performs a read on one of the server's characteristics.

The method checks which characteristic is being read by comparing its UUID. If it's the moisture characteristic, it refreshes the characteristic value with the current moisture percentage, then responds with `GATT_SUCCESS` and the encoded value. If it's the interval characteristic, it encodes the current reporting interval into two bytes using `intToTwoBytes` and sends that back. If it's the battery level characteristic, it encodes the current battery percentage into a single byte and returns it. If the UUID does not

The `onCharacteristicWriteRequest` method handles writes from the client. In this implementation, only the reporting interval characteristic is writable. When a write targets this characteristic, the code decodes the two byte value sent by the client into an integer by reconstructing it from the low and high bytes. It updates the internal `reportingIntervalSec` field, logs the new interval, and stores the received bytes in the characteristic so that future reads return the updated value. If the client requested a response, the server sends back a success status and echoes the written value.

Finally, `onDescriptorWriteRequest` is called when a client writes to a descriptor, typically the Client Characteristic Configuration Descriptor that controls notifications. The code checks whether the descriptor belongs to the moisture or battery characteristic and logs that notifications have been enabled for the corresponding data source. If a response is needed, it sends back `GATT_SUCCESS`.

Altogether, this callback turns the server into a live plant monitor that can answer real time read requests, accept configuration updates, and honor notification subscriptions for moisture and battery level.

We now have a fully functioning GATT Server that supports read and write operations, and can also send notifications for moisture and battery when needed.

To simulate notifications, the server can periodically update values and call `notifyCharacteristicChanged`:

```
    public void simulateSensorUpdate(BluetoothDevice device) {
        // Simulate moisture dropping slightly
```

This method simulates a live update to the soil moisture sensor and demonstrates how a GATT Server sends notifications to a connected BLE client.

It decreases the current moisture reading by one percent, ensuring the value never falls below zero using `Math.max`. After adjusting the simulated value, the method stores the updated moisture value inside the moisture characteristic using `setValue`, which prepares the new data to be transmitted.

It then calls `notifyCharacteristicChanged`, which sends a BLE notification packet to the specified connected device, telling the client that the characteristic value has changed and delivering the new moisture reading immediately.

The final parameter `false` indicates that this is a notification rather than an indication, which means the server does not require an acknowledgment from the client. This method would typically be called on a timer or triggered by real sensor hardware, allowing the client application to receive continuous updates in real time without repeatedly polling the server.

Implementing the GATT Client in Java

On the Android client side, we connect to the plant monitor, discover services, then interact with the three characteristics.

First, we'll discover the services and store references.

```
private final UUID PLANT_SERVICE_UUID =
        UUID.fromString("12345678-1234-5678-1234-56789abc0001");
private final UUID MOISTURE_CHAR_UUID =
        UUID.fromString("12345678-1234-5678-1234-56789abc0002");
private final UUID INTERVAL_CHAR_UUID =
        UUID.fromString("12345678-1234-5678-1234-56789abc0003");

private final UUID BATTERY_SERVICE_UUID =
        UUID.fromString("0000180F-0000-1000-8000-00805F9B34FB");
private final UUID BATTERY_LEVEL_UUID =
        UUID.fromString("00002A19-0000-1000-8000-00805F9B34FB");

public void connect(Context context, BluetoothDevice device) {
    bluetoothGatt = device.connectGatt(context, false, gattCallback);
}

private final BluetoothGattCallback gattCallback = new BluetoothGattCallback() {

    @Override
    public void onConnectionStateChange(BluetoothGatt gatt,
                                        int status,
                                        int newState) {
        if (newState == BluetoothProfile.STATE_CONNECTED) {
            Log.d(TAG, "Connected. Discovering services.");
            gatt.discoverServices();
        }
    }

    @Override
    public void onServicesDiscovered(BluetoothGatt gatt, int status) {
        Log.d(TAG, "Services discovered.");

        // Read current moisture and battery once
        readMoisture(gatt);
        readBatteryLevel(gatt);

        // Enable notifications
        enableMoistureNotifications(gatt);
    }
```

This class represents the Bluetooth Low Energy client side of the smart plant monitor example. It holds a `BluetoothGatt` reference that represents the active connection to the BLE server device.

interval, as well as the standard Battery Service and Battery Level characteristic.

The `connect` method starts the BLE connection by calling `device.connectGatt`, passing in the Android `Context`, a flag indicating no automatic reconnection, and a `BluetoothGattCallback` instance that will receive connection and data events.

Inside the callback, `onConnectionStateChange` is called whenever the connection state changes. When the new state indicates that the device is connected, the client logs this and calls `discoverServices` to request the full list of GATT services from the server.

Once the service discovery procedure completes, `onServicesDiscovered` is triggered. In this method, the client logs that services have been discovered, then immediately reads the current values of the moisture and battery level using helper methods `readMoisture` and `readBatteryLevel`, and finally enables notifications for moisture updates using `enableMoistureNotifications`.

Together, these steps mean that as soon as the client connects to a plant monitor device, it learns what services are available, fetches one time snapshots of important values, and subscribes to real time updates for the most important sensor – which in this case is soil moisture.

Now, we'll define methods for reading moisture and battery.

```
private void readMoisture(BluetoothGatt gatt) {
    BluetoothGattService service = gatt.getService(PLANT_SERVICE_UUID);
    if (service != null) {
        BluetoothGattCharacteristic ch =
```

```
        }
    }
}

private void readBatteryLevel(BluetoothGatt gatt) {
    BluetoothGattService service = gatt.getService(BATTERY_SERVICE_UUID);
    if (service != null) {
        BluetoothGattCharacteristic ch =
                service.getCharacteristic(BATTERY_LEVEL_UUID);
        if (ch != null) {
            gatt.readCharacteristic(ch);
        }
    }
}

@Override
public void onCharacteristicRead(BluetoothGatt gatt,
                                 BluetoothGattCharacteristic characteristic
                                 int status) {
    if (status == BluetoothGatt.GATT_SUCCESS) {
        if (MOISTURE_CHAR_UUID.equals(characteristic.getUuid())) {
            int moisture = characteristic.getIntValue(
                    BluetoothGattCharacteristic.FORMAT_UINT8, 0);
            Log.d(TAG, "Soil moisture: " + moisture + " percent");
        } else if (BATTERY_LEVEL_UUID.equals(characteristic.getUuid())) {
            int battery = characteristic.getIntValue(
                    BluetoothGattCharacteristic.FORMAT_UINT8, 0);
            Log.d(TAG, "Battery level: " + battery + " percent");
        } else if (INTERVAL_CHAR_UUID.equals(characteristic.getUuid())) {
            int interval = characteristic.getIntValue(
                    BluetoothGattCharacteristic.FORMAT_UINT16, 0);
            Log.d(TAG, "Reporting interval: " + interval + " sec");
        }
    }
}
```

These methods handle reading values from the smart plant monitor GATT Server. The `readMoisture` method retrieves the Plant Monitor Service using its UUID, then looks up the soil moisture characteristic inside it. If the characteristic is found, it sends a read request using `gatt.readCharacteristic`, which asks the server to return the current moisture value.

triggered. The method first checks whether the read was successful by confirming that the status equals `GATT_SUCCESS`. It then determines which characteristic was read by comparing UUIDs.

If the response is for the moisture characteristic, it decodes the value from a single byte into an integer percentage and logs it. If it is the battery characteristic, it similarly extracts the single byte battery percentage and logs that value. If the interval characteristic was read, it decodes two bytes into a 16 bit integer and logs the reporting interval in seconds.

This read flow provides the client with a snapshot of the current sensor and configuration values immediately after connecting, before monitoring changes through notifications.

Next, we'll enable notifications for moisture so that the app receives updates when it changes.

```
private void enableMoistureNotifications(BluetoothGatt gatt) {
    BluetoothGattService service = gatt.getService(PLANT_SERVICE_UUID);
    if (service != null) {
        BluetoothGattCharacteristic ch =
                service.getCharacteristic(MOISTURE_CHAR_UUID);
        if (ch != null) {
            gatt.setCharacteristicNotification(ch, true);

            BluetoothGattDescriptor descriptor =
                    ch.getDescriptor(UUID.fromString(
                            "00002902-0000-1000-8000-00805F9B34FB"));

            if (descriptor != null) {
                descriptor.setValue(
                        BluetoothGattDescriptor.ENABLE_NOTIFICATION_VALUE);
                gatt.writeDescriptor(descriptor);
            }
        }
    }
```

```
                                        BluetoothGattCharacteristic characteris
            if (MOISTURE_CHAR_UUID.equals(characteristic.getUuid())) {
                int moisture = characteristic.getIntValue(
                        BluetoothGattCharacteristic.FORMAT_UINT8, 0);
                Log.d(TAG,
                        "Soil moisture update: " + moisture + " percent");
            }
        }
    };
```

This code enables live moisture updates through notifications and handles them when they arrive.

The `enableMoistureNotifications` method first retrieves the Plant Monitor Service, then obtains the moisture characteristic using its UUID. If the characteristic is available, it calls `setCharacteristicNotification` with `true`, which tells the Android BLE stack to start listening for notifications on that characteristic.

But enabling notification support locally is not enough because the GATT specification requires that the client also write to the associated descriptor known as the Client Characteristic Configuration Descriptor, or CCCD, identified by the standard UUID `0x2902`. The method retrieves this descriptor, sets its value to `ENABLE_NOTIFICATION_VALUE`, and writes it using `writeDescriptor`, which sends a request over the air to the server to enable notifications on the device side. Once this configuration is complete, updates are delivered whenever the characteristic value changes.

The `onCharacteristicChanged` callback is triggered automatically each time the server pushes a new moisture reading. The method checks that the changed characteristic is the moisture characteristic by comparing UUIDs, extracts the soil moisture percentage from a single byte using `getIntValue`, and logs the updated value. This allows the client app to receive real time

dashboards or notification alerts.

Finally, the client can write a new reporting interval, for example changing from 60 seconds to 30 seconds.

```
public void writeReportingInterval(int newIntervalSec) {
    if (bluetoothGatt == null) return;

    BluetoothGattService service =
            bluetoothGatt.getService(PLANT_SERVICE_UUID);
    if (service != null) {
        BluetoothGattCharacteristic ch =
                service.getCharacteristic(INTERVAL_CHAR_UUID);
        if (ch != null) {
            byte[] data = new byte[2];
            data[0] = (byte) (newIntervalSec & 0xFF);
            data[1] = (byte) ((newIntervalSec >> 8) & 0xFF);
            ch.setValue(data);
            bluetoothGatt.writeCharacteristic(ch);
        }
    }
}
```

This method allows the BLE client to update the reporting interval setting on the smart plant monitor by writing a new value to the interval characteristic on the GATT Server.

It first checks whether the `bluetoothGatt` object is valid, since no write can occur before a connection is established. It retrieves the Plant Monitor Service using its UUID and then looks up the reporting interval characteristic inside that service.

If the characteristic exists, the method converts the new interval value from an integer into a two byte array, placing the least significant byte first and

characteristic's new value and then calls `writeCharacteristic`, which sends a write request over the air to the server. When the server processes the command in its corresponding write request handler, it will update its internal interval value and acknowledge the change.

This method demonstrates how configuration settings are written from a BLE client to a BLE device, enabling interactive control of behavior instead of only reading sensor values.

With this design, our smart plant monitor system is complete. The GATT Server exposes well-defined services and characteristics. The Android client connects, discovers, reads, writes, and subscribes to notifications. The concept is always the same: services group features. Characteristics hold data and behavior. Clients manipulate characteristics. Servers store and protect them.

Once you can design and code such a profile end to end, you are effectively using GATT the way real products do. The same pattern scales to complex devices like glucose monitors, smart locks, smart glasses, and industrial sensors.

Conclusion

GATT is the foundation that makes Bluetooth Low Energy communication understandable and reliable. It transforms raw radio signals into meaningful structured information through the use of services and characteristics. Once you understand that every BLE device exposes a database of values that a client can read, write, or subscribe to, the entire system becomes logical instead of mysterious.

characteristics inside services.

By examining both sides of the communication, the GATT Server and the GATT Client, and by walking through real Java code examples for reading, writing, and receiving notifications, you now have the practical knowledge needed to build and debug real BLE applications. You saw how to define custom services and characteristics, how to interpret data formats, how to enable notifications for dynamic sensor updates, and how to organize a complete device profile using a realistic example in the plant monitor project.

Everything in Bluetooth Low Energy development begins with understanding GATT at this level. Once you are comfortable designing and interacting with services and characteristics, you can confidently move into more advanced topics such as secure pairing and bonding, throughput tuning using MTU and connection interval, power optimization, OTA firmware updates, and tools like nRF Connect and HCI log analysis.

The best way to strengthen what you learned is to build something hands on. Even a simple read and write test project will help the concepts become intuitive.

Mastering GATT is the first major step toward professional Bluetooth development. Every complex system built with BLE, from consumer wearables to medical devices and smart home automation, sits on top of this technology. Now that you understand the structure and communication model, you are ready to explore more sophisticated capabilities and create your own applications with confidence.

If this article was helpful, share it.

Learn to code for free. freeCodeCamp's open source curriculum has helped more than 40,000 people get jobs as developers. Get started

Trending Books and Handbooks

REST APIs	Clean Code	TypeScript
JavaScript	AI Chatbots	Command Line
GraphQL APIs	CSS Transforms	Access Control
REST API Design	PHP	Java
Linux	React	CI/CD
Docker	Golang	Python
Node.js	Todo APIs	JavaScript Classes
Front-End Libraries	Express and Node.js	Python Code Examples
Clustering in Python	Software Architecture	Programming Fundamentals
Coding Career Preparation	Full-Stack Developer Guide	Python for JavaScript Devs

Mobile App

Download on the App Store

GET IT ON Google Play